

PyDiag

Test Diagnostic Python

Gestionnaire de Notes d'Étudiants

Notions évaluées

Variables et syntaxe, boucles et fonctions, structures de données

Table des matières

Contexte	2
Mission de l'apprenant	2
1 Modules à développer	2
1.1 Module 1 — Variables et syntaxe (15 min — 15 pts)	2
1.2 Module 2 — Boucles et fonctions (25 min — 25 pts)	3
1.3 Module 3 — Structures de données (25 min — 25 pts)	5
2 Section Bonus — Défis avancés (optionnel, hors temps imparti)	7
3 Règles de Gestion	8
4 Modélisation des données	8
5 Jeu de données de test (exemple global)	9
6 Structure attendue du script	9
7 Livrables attendus	10

Contexte

Dans le cadre de l'évaluation diagnostique des apprenants, **PyDiag** est un mini-projet Python permettant de vérifier la maîtrise des fondamentaux du langage à travers un cas d'usage unique et progressif : un **gestionnaire de notes d'étudiants**.

L'apprenant construit une application en console qui permet de saisir les notes d'un groupe d'étudiants, de calculer leurs moyennes et mentions, de les classer, puis de générer des statistiques de groupe.

L'objectif est d'évaluer la capacité de l'apprenant à :

- Manipuler variables, types et syntaxe Python de façon rigoureuse
- Structurer un traitement avec boucles et fonctions réutilisables
- Choisir et manipuler les structures de données adaptées (listes, tuples, dictionnaires, sets)
- Aller au-delà des cas simples : gérer les cas limites, les données incomplètes, les erreurs de saisie

Mission de l'apprenant

En tant que développeur, vous devez concevoir un script Python unique et cohérent, en respectant les bonnes pratiques : nommage clair, fonctions courtes à responsabilité unique, code lisible et testé au fur et à mesure.

Le script devra :

- Représenter chaque étudiant et ses notes de façon structurée
- Calculer moyennes, mentions et statistiques de groupe via des fonctions dédiées
- Trier, filtrer et regrouper les étudiants selon leurs résultats
- Gérer proprement les cas limites (liste de notes vide, doublons de noms, valeurs aux frontières des mentions)

1 Modules à développer

1.1 Module 1 — Variables et syntaxe (15 min — 15 pts)

Module 1 : Variables et syntaxe

- ☐ Définir un étudiant : nom, prénom, liste de 3 notes
- ☐ Calculer sa moyenne « à la main » (sans `sum()` ni `len()`)
- ☐ Afficher un résumé formaté avec des f-strings, aligné proprement (ex. `:>6.2f`)
- ☐ Respecter les conventions de nommage Python (snake_case, constantes en MAJUSCULES si besoin)

Exemple — Données & résultat attendu**Données d'entrée :**

```
nom = "Karim"
prenom = "Ben Ali"
notes = [12, 15, 9]
```

Sortie attendue (après affichage formaté) :

```
Karim moyenne : 12.00
```

Indice de lecture : `>:6.2f` veut dire « nombre flottant, 2 décimales, aligné à droite sur 6 caractères ». C'est ce type de détail de formatage qui est évalué, pas seulement le résultat du calcul.

Défis supplémentaires :

- ☐ Demander nom, prénom et 3 notes via `input()`, en convertissant correctement les types (`str` → `float`)
- ☐ Valider que chaque note saisie est bien un nombre avant de continuer (sans encore lever d'exception personnalisée — juste une vérification simple)

Exemple — Données & résultat attendu**Saisie simulée par l'utilisateur :**

```
Entrez une note : 17
Entrez une note : abc
Entrez une note : 14
```

Comportement attendu :

```
Note acceptee : 17.0
Ce n'est pas un nombre valide, reessayez.
Note acceptee : 14.0
```

Rappel : `input()` renvoie toujours du texte (`str`), même si l'utilisateur tape un chiffre. La conversion et la vérification sont à votre charge.

1.2 Module 2 — Boucles et fonctions (25 min — 25 pts)**Module 2 : Boucles et fonctions**

- ☐ Fonction `calculer_moyenne(notes)` → retourne la moyenne d'une liste de notes
- ☐ Fonction `appreciation(moyenne)` → retourne une mention selon des tranches définies
- ☐ Boucle `for` traitant une liste de plusieurs étudiants, affichant nom / moyenne / mention pour chacun
- ☐ Identification du meilleur et du moins bon étudiant via la boucle

Exemple — Données & résultat attendu**Données d'entrée (calculer_moyenne) :**

```
notes = [12, 15, 9]
```

Résultat attendu : 12.0

Exemple — Données & résultat attendu**Données d'entrée (appreciation) :**

```
valeurs_test = [9.9, 10.0, 11.9, 12.0, 15.9, 16.0, 20.0]
```

Sortie attendue (mention associée à chaque valeur) :

```
9.9 -> Insuffisant
10.0 -> Passable
11.9 -> Passable
12.0 -> Bien
15.9 -> Bien
16.0 -> Tres bien
20.0 -> Tres bien
```

Ce test vérifie que vos comparaisons (<, <=) sont posées exactement aux bonnes frontières.

Exemple — Données & résultat attendu**Données d'entrée (boucle + meilleur/moins bon) :**

```
etudiants = [
    {"nom": "Karim", "notes": [12, 15, 9]},
    {"nom": "Sara", "notes": [18, 17, 16]},
    {"nom": "Lina", "notes": [6, 8, 5]},
]
```

Sortie attendue :

```
Karim 12.00 Passable
Sara 17.00 Tres bien
Lina 6.33 Insuffisant
Meilleur etudiant : Sara
Moins bon etudiant : Lina
```

Défis supplémentaires :

- ☐ Fonction `calculer_moyenne_ponderee(notes, coefficients)` qui applique des coefficients différents par matière
- ☐ Fonction `moyenne_groupe(etudiants)` qui calcule la moyenne générale de toute la classe
- ☐ Gérer le cas d'une liste de notes vide sans provoquer de `ZeroDivisionError` (retourner `None` ou 0 avec un message clair)
- ☐ Fonction récursive `somme_recursive(notes)` pour calculer la somme des notes

Exemple — Données & résultat attendu**Données d'entrée (calculer_moyenne_ponderee) :**

```
notes = [14, 10, 18] # Maths, Francais, Sport
coefficients = [3, 2, 1]
```

Résultat attendu : 15.33

Exemple — Données & résultat attendu**Données d'entrée (moyenne_groupe) :**

```
etudiants = [  
    {"nom": "Karim", "notes": [12, 15, 9]},  
    {"nom": "Sara", "notes": [18, 17, 16]},  
    {"nom": "Lina", "notes": [6, 8, 5]},  
]
```

Résultat attendu : 11.44 (moyenne des 3 moyennes individuelles)**Exemple — Données & résultat attendu****Données d'entrée (cas limite — liste vide) :**

```
notes = []
```

Comportement attendu :

Attention : aucune note fournie.

(pas de plantage, pas de ZeroDivisionError)

Exemple — Données & résultat attendu**Données d'entrée (somme_recursive) :**

```
notes = [12, 15, 9, 18]
```

Résultat attendu : 54**1.3 Module 3 — Structures de données (25 min — 25 pts)****Module 3 : Structures de données**

- ☐ Construction d'un dictionnaire `resultats` : `{nom: {"moyenne": ..., "mention": ...}}`
- ☐ Classement des étudiants par moyenne décroissante (`sorted()` + `key`)
- ☐ Extraction des étudiants en échec (`moyenne < 10`) dans une liste de tuples (`nom, moyenne`)
- ☐ Au moins une compréhension de liste ou de dictionnaire utilisée

Exemple — Données & résultat attendu**Données d'entrée (construire_resultats) :**

```
etudiants = [  
    {"nom": "Karim", "notes": [12, 15, 9]},  
    {"nom": "Sara", "notes": [18, 17, 16]},  
    {"nom": "Lina", "notes": [6, 8, 5]},  
]
```

Résultat attendu (dictionnaire resultats) :

```
{  
    "Karim": {"moyenne": 12.0, "mention": "Bien"},  
    "Sara": {"moyenne": 17.0, "mention": "Tres bien"},  
    "Lina": {"moyenne": 6.3, "mention": "Insuffisant"},  
}
```

Exemple — Données & résultat attendu**Données d'entrée (classer_par_moyenne et etudiants_en_echec) :**

```
resultats = {  
    "Karim": {"moyenne": 12.0, "mention": "Bien"},  
    "Sara": {"moyenne": 17.0, "mention": "Tres bien"},  
    "Lina": {"moyenne": 8.7, "mention": "Insuffisant"},  
}
```

Classement attendu (décroissant) :

```
1. Sara - 17.0  
2. Karim - 12.0  
3. Lina - 8.7
```

Liste des étudiants en échec attendue :

```
[("Lina", 8.7)]
```

Défis supplémentaires :

- ☐ Regrouper les étudiants par mention dans un dictionnaire {"Bien": [...], "Passable": [...], ...}
- ☐ Utiliser un **set** pour détecter s'il y a des doublons de noms dans la liste des étudiants
- ☐ Fusionner deux groupes d'étudiants (deux dictionnaires) sans écraser les données si un nom existe dans les deux

Exemple — Données & résultat attendu**Données d'entrée (regrouper_par_mention) :**

```
resultats = {
    "Karim": {"moyenne": 12.0, "mention": "Bien"},
    "Sara": {"moyenne": 17.0, "mention": "Tres bien"},
    "Lina": {"moyenne": 8.7, "mention": "Insuffisant"},
    "Nadia": {"moyenne": 13.5, "mention": "Bien"},
}
```

Résultat attendu :

```
{
    "Bien": ["Karim", "Nadia"],
    "Tres bien": ["Sara"],
    "Insuffisant": ["Lina"],
}
```

Exemple — Données & résultat attendu**Données d'entrée (détection de doublon) :**

```
noms = ["Karim", "Sara", "Lina", "Karim"]
```

Sortie attendue :

```
Attention, il y a des doublons !
```

Exemple — Données & résultat attendu**Données d'entrée (fusion de deux groupes) :**

```
groupe_a = {
    "Karim": {"moyenne": 12.0, "mention": "Bien"},
}
groupe_b = {
    "Karim": {"moyenne": 15.0, "mention": "Bien"},
    "Sara": {"moyenne": 17.0, "mention": "Tres bien"},
}
```

Comportement attendu : Sara est ajoutée normalement au résultat fusionné. Pour Karim, présent dans les deux groupes, les données ne doivent pas être écrasées silencieusement — vous devez choisir et documenter une stratégie (conserver les deux valeurs, refuser la fusion pour ce nom, ou signaler le conflit).

2 Section Bonus — Défis avancés (optionnel, hors temps imparti)

Pour les apprenants ayant terminé les 3 modules avant la fin des 65 minutes.

Défis avancés (B1 à B2)

#	Défi	Notion combinée
B1	Ajouter une interface console en boucle (while True) avec un menu : Ajouter étudiant / Afficher classement / Quitter	Boucles, fonctions, structures

Exemple — B1 : menu console

Entrée utilisateur simulée :

```
Choix : 1
Nom : Yassine
Notes : 14 12 16
Choix : 2
Choix : 3
```

Sortie attendue :

```
1. Ajouter etudiant
2. Afficher classement
3. Quitter
Etudiant Yassine ajoute.
1. Yassine - 14.0
Au revoir !
```

3 Règles de Gestion

Notes et étudiants

- Chaque étudiant possède exactement 3 notes, comprises entre 0 et 20
- La moyenne est calculée sur ces 3 notes, arrondie à 1 décimale
- Un étudiant est en échec si sa moyenne est strictement inférieure à 10
- Un nom d'étudiant est considéré unique dans le groupe (pas de doublon accepté)

Mentions

Moyenne	Mention
< 10	Insuffisant
$10 \leq m < 12$	Passable
$12 \leq m < 16$	Bien
≥ 16	Très bien

4 Modélisation des données

Structure resultats

Champ	Type	Description
nom	str (clé)	Nom de l'étudiant
moyenne	float	Moyenne calculée sur les 3 notes
mention	str	Mention déduite de la moyenne

5 Jeu de données de test (exemple global)

Voici un jeu de données prêt à copier-coller pour tester l'ensemble de votre script. Il couvre volontairement plusieurs **cas limites** : une moyenne pile sur une frontière de mention, un nom en doublon, et une liste de notes vide.

Données d'entrée

```
etudiants_test = [
    {"nom": "Karim", "notes": [12, 8, 16]},
    {"nom": "Sara", "notes": [18, 17, 19]},
    {"nom": "Lina", "notes": [5, 6, 4]},
    {"nom": "Youssef", "notes": [10, 10, 10]},
    {"nom": "Nadia", "notes": [16, 16, 16]},
    {"nom": "Karim", "notes": [11, 12, 13]},
    {"nom": "Hicham", "notes": []},
]
```

Résultat attendu par étudiant

Nom	Moyenne	Comportement / mention
Karim	12.0	Bien
Sara	18.0	Tres bien
Lina	5.0	Insuffisant
Youssef	10.0	Passable (frontière basse)
Nadia	16.0	Tres bien (frontière haute)
Karim (2 ^e)	—	Doublon de nom à détecter
Hicham	—	Liste vide : pas de ZeroDivisionError

6 Structure attendue du script

```
gestion_notes.py
|
|-- calculer_moyenne(notes)
|-- calculer_moyenne_ponderee(notes, coefficients) # bonus
|-- appreciation(moyenne)
|-- construire_resultats(etudiants)
|-- classer_par_moyenne(resultats)
|-- etudiants_en_echec(resultats)
|-- regrouper_par_mention(resultats) # bonus
|-- moyenne_groupe(etudiants) # bonus
|-- somme_recursive(notes) # bonus
```

7 Livrables attendus

1. Un script `.py` unique, fonctionnel de bout en bout pour les 3 modules obligatoires
2. Un jeu de données de test d'au moins 5 étudiants, incluant au moins un cas limite (moyenne à la frontière d'une mention, liste de notes vide, doublon de nom)
3. Une exécution démontrant : calcul, classement, filtrage des étudiants en échec, gestion des cas limites
4. (Optionnel) Les défis bonus réalisés, clairement identifiés dans le code ou en commentaire